

QA Automation Engineering Intern

Case Study Response

Candidate: Sahil Kute

Role: QA Automation Engineering Intern

B2B SaaS Platform Testing - Multi-Platform Automation

Assumptions

Since some requirements were intentionally left incomplete, I made the following assumptions while designing the solution:

- Authentication is performed using email and password.
- Users who require 2FA are provided with test accounts where 2FA can either be bypassed or completed using test OTPs.
- API authentication is handled using Bearer Tokens.
- Each tenant has its own unique Tenant ID.
- Test environment contains dedicated automation users and test data.
- BrowserStack is assumed to be available for cross-browser and mobile test execution.

Part 1 - Debugging Flaky Test Code

1. Identify Flakiness Issues

The existing Playwright tests may fail intermittently due to several reasons.

Issue 1: No waiting after login

```
page.click("#login-btn")  
assert page.url == "https://app.workflowpro.com/dashboard"
```

Immediately after clicking Login, the application may still be processing authentication.

The dashboard may not have loaded completely when the assertion executes.

Result:

- Passes locally sometimes
- Fails in CI

Issue 2: URL assertion is too strict

```
assert page.url == ...
```

Problems:

- Redirect may take time.
- Query parameters may be appended.
- Trailing slash may exist.
- Application may redirect through SSO.

Better approach:

```
expect(page).to_have_url(re.compile(".*dashboard"))
```

Issue 3: Dynamic Dashboard Loading

The case study states that dashboard elements are loaded dynamically.

```
assert page.locator(".welcome-message").is_visible()
```

The welcome message may appear several seconds after navigation.

Without waiting, the assertion becomes flaky.

Issue 4: Browser launched with default configuration

```
browser = p.chromium.launch()
```

Missing configuration:

- headless mode
- viewport
- timeout
- tracing
- screenshots

CI machines often have different screen resolutions.

Issue 5: No handling of 2FA

The requirements mention:

Login process includes 2FA for some users.

Current test assumes every login immediately reaches dashboard.

This causes failures for users requiring OTP verification.

Issue 6: Multi-tenant project loading

```
projects = page.locator(".project-card").all()
```

Immediately fetching all project cards may return an empty list because data is still loading.

Issue 7: Weak validation

Current validation:

```
assert "Company2" in project.text_content()
```

Problems:

- Doesn't verify project count.
- Doesn't verify tenant ID.
- Doesn't verify unauthorized projects are absent.

Issue 8: Missing Cleanup

If test data is created during execution, it remains in the environment.

Future executions may become inconsistent.

Issue 9: Browser not closed on failure

If any assertion fails,

```
browser.close()
```

is never executed.

This can leave browser instances running in CI.

Issue 10: Hardcoded Credentials

admin@company1.com

password123

Credentials should never be stored directly in source code.

Instead, they should come from environment variables or configuration files.

2. Why do these failures occur more often in CI/CD?

Tests usually run successfully on a developer's machine but fail in CI because the execution environment is different.

Local Machine	CI/CD Environment
Faster hardware	Shared resources
Stable internet	Variable network latency
Same browser every time	Multiple browser versions
Fixed screen size	Different resolutions
User interaction possible	Fully headless
Less parallel execution	Multiple tests run simultaneously

Because of these differences, tests that rely on timing instead of application state become unreliable.

3. Root Cause Summary

The primary causes of flaky tests are:

- Missing synchronization
- Dynamic UI loading
- Network delays
- Different browser speeds
- Hardcoded assumptions
- Poor locator strategy

- Missing error handling
- Test data dependency

4. Corrected Playwright Test

```
import re

import pytest

from playwright.sync_api import sync_playwright, expect


def test_user_login():
    with sync_playwright() as p:
        browser = p.chromium.launch(
            headless=True
        )

        page = browser.new_page(
            viewport={"width": 1440, "height": 900}
        )

        page.goto(
            "https://app.workflowpro.com/login",
            wait_until="networkidle"
        )

        page.fill("#email", "admin@company1.com")
        page.fill("#password", "password123")

        page.click("#login-btn")

        # Handle optional 2FA
        if page.locator("#otp").is_visible(timeout=3000):
            page.fill("#otp", "123456")
```

```
page.click("#verify-btn")

expect(page).to_have_url(
    re.compile(".*dashboard")
)

expect(
    page.locator(".welcome-message")
).to_be_visible(timeout=10000)

browser.close()
```

Improved Multi-Tenant Test

```
def test_multi_tenant_access():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)

        page = browser.new_page()

        page.goto(
            "https://app.workflowpro.com/login",
            wait_until="networkidle"
        )

        page.fill("#email", "user@company2.com")
        page.fill("#password", "password123")

        page.click("#login-btn")

        expect(page).to_have_url(
            re.compile(".*dashboard")
```

```
)

expect(
    page.locator(".project-card").first
).to_be_visible(timeout=10000)

projects = page.locator(".project-card")

total = projects.count()

assert total > 0

for i in range(total):
    text = projects.nth(i).text_content()
    assert "Company2" in text
    assert "Company1" not in text

browser.close()
```

5. Reliability Improvements

To further improve test stability, I would:

- Use Page Object Model instead of writing locators inside test cases.
- Store credentials in environment variables.
- Capture screenshots on failure.
- Enable Playwright tracing for debugging.
- Retry only flaky infrastructure failures, not assertion failures.
- Use dedicated automation test accounts.
- Clean up test data after execution.
- Execute tests in parallel with isolated test data.
- Generate HTML and Allure reports for better visibility.

Part 2 - Test Framework Design

Objective

The goal is to design a scalable, maintainable, and reusable automation framework for a B2B SaaS application that supports:

- Web automation (Chrome, Firefox, Safari)
- Mobile automation (iOS, Android)
- Multiple tenants
- Multiple user roles
- API testing
- BrowserStack integration
- CI/CD execution

1. Framework Structure

To keep the framework modular and easy to maintain, I would follow the Page Object Model (POM) along with a layered project structure.

automation-framework/

```
|
|
| └─ config/
|   └─ config.py
|   └─ environments.json
|   └─ browserstack.json
|
| └─ pages/
|   └─ base_page.py
|   └─ login_page.py
|   └─ dashboard_page.py
|   └─ project_page.py
|   └─ mobile_project_page.py
|
| └─ tests/
|   └─ web/
```



```
| | | └─ test_login.py
| | | └─ test_projects.py
| | | └─ test_roles.py
| |
| | └─ api/
| | | └─ test_project_api.py
| | | └─ test_auth_api.py
| |
| | └─ integration/
| | | └─ test_project_creation_flow.py
| |
| | └─ utils/
| | | └─ api_client.py
| | | └─ logger.py
| | | └─ test_data.py
| | | └─ browser_factory.py
| | | └─ helpers.py
| |
| | └─ fixtures/
| | | └─ conftest.py
| |
| | └─ reports/
| |
| | └─ screenshots/
| |
| | └─ requirements.txt
| |
| | └─ pytest.ini
```

Why this structure?

Each folder has a single responsibility.

Folder	Purpose
config	Stores environment-specific configurations
pages	Contains Page Object classes for UI automation
tests	Contains all test cases categorized by type
utils	Reusable helper functions and utilities
fixtures	Shared pytest fixtures for setup and teardown
reports	Test execution reports
screenshots	Failure screenshots for debugging

This separation improves readability and simplifies maintenance as the project grows.

2. Base Classes

Base Page

All page classes should inherit from a common BasePage.

It should contain reusable methods such as:

- `click()`
- `fill()`
- `wait_for_element()`
- `wait_for_page_load()`
- `take_screenshot()`
- `get_text()`

This prevents duplicate code across page classes.

Example:

```
class BasePage:
    def click(self, locator):
        self.page.locator(locator).click()
```

```
def fill(self, locator, value):  
    self.page.locator(locator).fill(value)  
  
def wait(self, locator):  
    self.page.locator(locator).wait_for()
```

3. Pytest Fixtures

To avoid repetitive setup and teardown code, the framework would use **pytest fixtures** defined in `conftest.py`. Fixtures can initialize the browser, create browser contexts, manage user authentication, and perform cleanup after test execution. This improves code reusability, keeps test cases concise, and ensures consistent test execution across the framework.

4. Page Object Model (POM)

Instead of writing locators inside test cases, each page should manage its own locators and actions.

Example:

LoginPage

```
class LoginPage:  
    def login(self, email, password):  
        self.page.fill("#email", email)  
        self.page.fill("#password", password)  
        self.page.click("#login-btn")
```

The test then becomes:

```
login.login(admin_email, password)  
dashboard.verify_dashboard_loaded()
```

Advantages

- Easier maintenance
- Reusable code
- Better readability
- Changes in UI require updates only in one place

5. Configuration Management

The framework should support multiple environments without modifying the test code.

Example configuration:

Development

QA

Staging

Production

Each environment should store:

- Base URL
- API URL
- Browser
- Timeout
- Tenant information

Instead of hardcoding values, configuration files or environment variables should be used.

Example:

BASE_URL

API_URL

BROWSER

HEADLESS

TENANT_ID

This allows the same test suite to run against different environments by simply changing the configuration.

6. Multiple Browser Support

The framework should support:

- Chrome
- Firefox

- Safari

A browser factory can initialize the required browser based on configuration.

Example:

```
browser = BrowserFactory.create(browser_name)
```

No changes are required in the test scripts when switching browsers.

7. Multi-Tenant Support

Since this is a multi-tenant SaaS application, the framework should separate tenant-specific data.

Example:

Tenant 1

Admin

Manager

Employee

Tenant 2

Admin

Manager

Employee

Each tenant should have:

- Separate credentials
- Separate API tokens
- Separate test data
- Separate expected results

This ensures tests validate tenant isolation correctly.

8. User Role Management

The framework should support multiple user roles such as:

- Admin
- Manager
- Employee

Instead of hardcoding credentials, role-based test data should be maintained.

Example:

```
login(get_admin())
```

```
login(get_manager())
```

```
login(get_employee())
```

This makes role-based permission testing easier and avoids duplication.

9. API Testing Layer

A separate API utility should handle all backend requests.

Responsibilities include:

- Authentication
- GET requests
- POST requests
- PUT requests
- DELETE requests

Example:

```
api.create_project()
```

```
api.delete_project()
```

```
api.get_projects()
```

This keeps API logic separate from UI tests and promotes code reuse.

10. BrowserStack Integration

BrowserStack enables execution on real browsers and mobile devices without maintaining physical hardware.

The framework should allow execution on:

Browsers

- Chrome
- Firefox

- Safari

Mobile Devices

- Android
- iPhone

BrowserStack credentials should be stored securely using environment variables rather than hardcoded in the codebase.

11. CI/CD Integration

The framework should integrate with CI/CD tools such as GitHub Actions or Jenkins.

Typical pipeline flow:

Developer pushes code

↓

CI Pipeline starts

↓

Install dependencies

↓

Run API Tests

↓

Run Web Tests

↓

Run Mobile Tests

↓

Generate Reports

↓

Publish Results

This ensures automated validation before deployment.

12. Test Data Management

Reliable automation depends on controlled and isolated test data.

Recommended practices:

- Use dedicated automation accounts.
- Generate unique project names during execution.
- Avoid sharing mutable data between parallel tests.
- Clean up test data after execution using APIs.
- Reset the environment if required.

This minimizes test failures caused by stale or conflicting data.

13. Reporting and Logging

To simplify debugging and analysis, the framework should generate:

- HTML execution reports
- Allure reports (if available)
- Failure screenshots
- Playwright traces
- Execution logs

These artifacts help identify failures quickly and improve troubleshooting.

14. Parallel Execution

To reduce execution time, tests should be capable of running in parallel.

While enabling parallel execution:

- Each test should have independent data.
- Browser sessions should remain isolated.
- Shared resources should be avoided.

This improves execution speed without affecting reliability.

15. Missing Requirements / Questions

Before implementing the framework, I would clarify the following points:

1. Is there a dedicated QA or staging environment for automation?
2. How are test users and credentials managed?
3. Can 2FA be bypassed for automation accounts?
4. Is there an API available for test data cleanup?
5. Which reporting framework is preferred (HTML, Allure, etc.)?

6. How many parallel executions are expected in CI?
7. Which browsers and mobile device versions are officially supported?
8. What is the expected execution frequency (per commit, nightly, release)?
9. Should performance and accessibility testing be included?
10. What are the acceptable execution time and failure thresholds?

Part 3 - API + UI Integration Test

Objective

The objective is to validate the complete project creation workflow by testing both the backend APIs and the user interface.

Instead of creating a project manually through the UI, the project is first created using the API. The UI is then used to verify that the data is displayed correctly. Finally, the same project is validated on a mobile device, and tenant isolation is verified to ensure that users from other companies cannot access the project.

This approach is faster, more reliable, and provides better coverage than relying only on UI testing.

Testing Strategy

The integration test consists of four stages:

1. **API Layer**
 - Create a new project using the backend API.
2. **Web UI Layer**
 - Log in through the web application.
 - Verify that the project appears correctly on the dashboard.
3. **Mobile Layer**
 - Open the application on a mobile device using BrowserStack.
 - Verify that the same project is visible.
4. **Security Validation**
 - Log in as a user from another tenant.
 - Confirm that the created project is not accessible.

This validates both functionality and security.

Test Flow

Start Test



Authenticate API



Create Project using API



Store Project ID



Login through Web UI



Verify Project Exists



Open Mobile Device (BrowserStack)



Verify Same Project



Login as Another Tenant



Verify Project is NOT Visible



Delete Test Data



End Test

Integration Test

```
import uuid
import requests
import re
from playwright.sync_api import sync_playwright, expect
```

```
BASE_URL = "https://app.workflowpro.com"
```

```
API_URL = "https://app.workflowpro.com/api/v1"
```

```
TOKEN = os.getenv("API_TOKEN")
```

```
TENANT_ID = "company1"
```

```
def test_project_creation_flow():
```

```
    # ----- Step 1 : Create project through API -----
```

```
    project_name = f"Automation-{uuid.uuid4().hex[:6]}"
```

```
    headers = {
        "Authorization": f"Bearer {TOKEN}",
        "X-Tenant-ID": TENANT_ID
    }
```

```
    payload = {
        "name": project_name,
        "description": "Automation Test Project",
        "team_members": []
    }
```

```
    response = requests.post(
        f"{API_URL}/projects",
        headers=headers,
        json=payload
```

```

)
assert response.status_code == 201
project_id = response.json()["id"]

# ----- Step 2 : Verify in Web UI -----

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto(
        f"{BASE_URL}/login",
        wait_until="networkidle"
    )
    page.fill("#email", "admin@company1.com")
    page.fill("#password", "password123")
    page.click("#login-btn")
    expect(page).to_have_url(
        re.compile(".*dashboard")
    )
    page.reload()
    expect(
        page.locator(f"text={project_name}")
    ).to_be_visible(timeout=10000)
    browser.close()

# ----- Step 3 : Mobile Validation -----

# BrowserStack execution would be configured here.
# Login on Android/iOS device
# Verify project_name is visible.

```

----- Step 4 : Tenant Isolation -----

with sync_playwright() as p:

```
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto(
        f'{BASE_URL}/login',
        wait_until="networkidle"
    )
    page.fill("#email", "admin@company2.com")
    page.fill("#password", "password123")
    page.click("#login-btn")
    expect(page).to_have_url(
        re.compile(".*dashboard")
    )
    assert page.locator(
        f"text={project_name}"
    ).count() == 0
    browser.close()
```

----- Step 5 : Cleanup -----

```
requests.delete(
    f'{API_URL}/projects/{project_id}',
    headers=headers
)
```

Handling Test Data

Proper test data management is important to ensure tests remain reliable and independent.

The following practices would be followed:

- Generate a unique project name using UUID to avoid conflicts.

- Use dedicated automation test accounts instead of production users.
- Store authentication tokens securely in environment variables.
- Clean up created projects after execution using APIs.
- Avoid sharing test data between parallel executions.

These practices reduce test failures caused by duplicate or stale data.

Cross-Platform Validation

The framework should support execution on multiple browsers and mobile devices.

Web Browsers

- Chrome
- Firefox
- Safari

Mobile Devices

- Android
- iPhone

Using BrowserStack, the same test can be executed on different operating systems and browser versions without modifying the test logic.

BrowserStack Integration

The framework would use BrowserStack for executing tests on real devices and browsers.

Example execution flow:

Local Test

↓

Push Code

↓

CI Pipeline

↓

BrowserStack

↓

Chrome

Firefox

Safari

Android

iPhone



Reports

This ensures compatibility across supported platforms while avoiding the cost of maintaining physical devices.

Tenant Isolation Validation

Tenant isolation is one of the most critical requirements for a multi-tenant SaaS application.

The following checks should be performed:

- Users should only access projects belonging to their own tenant.
- Projects created for Company 1 must not appear for Company 2.
- API requests should reject invalid tenant IDs.
- Authorization tokens should only access permitted tenant data.
- Direct URL access to another tenant's resources should be denied.

These validations help prevent data leakage between customers.

Edge Cases Considered

The integration test should also handle situations such as:

- Slow network responses.
- Temporary API failures.
- Authentication token expiration.
- Browser crashes.
- Mobile rendering delays.
- Empty project lists.
- Duplicate project names.
- Invalid tenant IDs.
- Server timeouts.
- Retry logic for transient failures.

Handling these cases improves the robustness of the automation suite.

Testing Approach

The proposed approach follows an **API-first testing strategy**.

Instead of creating data through the UI, test data is created directly using backend APIs. This reduces execution time, minimizes UI dependencies, and makes tests more reliable. The UI is then used only to verify that the application displays the expected information correctly.

Finally, mobile validation and tenant isolation checks ensure that the application behaves consistently across platforms while maintaining strict data separation between different customers.

Conclusion

The proposed integration test validates the complete workflow of project creation across API, web, and mobile layers. It also verifies tenant isolation, ensures proper cleanup of test data, and supports execution across multiple browsers and devices. This approach provides faster execution, higher reliability, and better coverage for a modern B2B SaaS application.

References / Tools Used

- Python
- pytest
- Playwright
- Requests (API Testing)
- BrowserStack (Conceptual Integration)
- Page Object Model (POM)
- Git & CI/CD Concepts

Final Note

This solution has been designed based on the information provided in the case study. Wherever requirements were intentionally incomplete, reasonable assumptions have been made following standard QA automation practices. The proposed framework emphasizes maintainability, scalability, and reliability while supporting multi-tenant testing, cross-platform validation, and CI/CD integration.